JavaScript Objects

« Previous Next Chapter »

JavaScript Objects

In JavaScript, objects are king. If you understand objects, you understand JavaScript.

In JavaScript, almost "everything" is an object.

- Booleans can be objects (or primitive data treated as objects)
- Numbers can be objects (or primitive data treated as objects)
- Strings can be objects (or primitive data treated as objects)
- Dates are always objects
- Maths are always objects
- Regular expressions are always objects
- Arrays are always objects
- Functions are always objects
- Objects are objects

In JavaScript, all values, except primitive values, are objects.

Primitive values are: strings ("John Doe"), numbers (3.14), true, false, null, and undefined.

Objects are Variables Containing Variables

JavaScript variables can contain single values:

Example

```
var person = "John Doe";
```

Try it Yourself »

Objects are variables too. But objects can contain many values.

The values are written as **name : value** pairs (name and value separated by a colon).

Example

```
var person = {firstName:"John", lastName:"Doe", age:50,
eyeColor:"blue"};
```

Try it Yourself »

A JavaScript object is a collection of **named values**

Object Properties

The named values, in JavaScript objects, are called **properties**.

Property	Value
firstName	John
lastName	Doe
age	50
eyeColor	blue

Objects written as name value pairs are similar to:

- Associative arrays in PHP
- Dictionaries in Python
- Hash tables in C
- Hash maps in Java
- Hashes in Ruby and Perl

Object Methods

Methods are **actions** that can be performed on objects.

Object properties can be both primitive values, other objects, and functions.

An **object method** is an object property containing a **function definition**.

Property	Value
firstName	John
lastName	Doe
age	50
eyeColor	blue
fullName	function() {return this.firstName + " " + this.lastName;}

JavaScript objects are containers for named values, called properties and methods.

You will learn more about methods in the next chapters.

Creating a JavaScript Object

With JavaScript, you can define and create your own objects.

There are different ways to create new objects:

- Define and create a single object, using an object literal.
- Define and create a single object, with the keyword new.
- Define an object constructor, and then create objects of the constructed type.

In ECMAScript 5, an object can also be created with the function Object.create().

Using an Object Literal

This is the easiest way to create a JavaScript Object.

Using an object literal, you both define and create an object in one statement.

An object literal is a list of name:value pairs (like age:50) inside curly braces {}.

The following example creates a new JavaScript object with four properties:

Example

```
var person = {firstName:"John", lastName:"Doe", age:50,
eyeColor:"blue"};
```

Try it Yourself »

Spaces and line breaks are not important. An object definition can span multiple lines:

Example

```
var person = {
    firstName:"John",
    lastName:"Doe",
    age:50,
    eyeColor:"blue"
};
```

Try it Yourself »

Using the JavaScript Keyword new

The following example also creates a new JavaScript object with four properties:

Example

```
var person = new Object();
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";
```

Try it Yourself »

The two examples above do exactly the same. There is no need to use new Object().
For simplicity, readability and execution speed, use the first one (the object literal method).

Using an Object Constructor

The examples above are limited in many situations. They only create a single object.

Sometimes we like to have an "object type" that can be used to create many objects of one type.

The standard way to create an "object type" is to use an object constructor function:

Example

```
function person(first, last, age, eye) {
    this.firstName = first;
    this.lastName = last;
    this.age = age;
    this.eyeColor = eye;
}
var myFather = new person("John", "Doe", 50, "blue");
var myMother = new person("Sally", "Rally", 48, "green");
```

Try it yourself »

The above function (person) is an object constructor.

Once you have an object constructor, you can create new objects of the same type:

```
var myFather = new person("John", "Doe", 50, "blue");
var myMother = new person("Sally", "Rally", 48, "green");
```

The *this* Keyword

In JavaScript, the thing called **this**, is the object that "owns" the JavaScript code.

The value of **this**, when used in a function, is the object that "owns" the function.

The value of **this**, when used in an object, is the object itself.

The **this** keyword in an object constructor does not have a value. It is only a substitute for the new object.

The value of **this** will become the new object when the constructor is used to create an object.

Note that **this** is not a variable. It is a keyword. You cannot change the value of **this**.

Built-in JavaScript Constructors

JavaScript has built-in constructors for native objects:

Example

```
var x1 = new Object();    // A new Object object
var x2 = new String();    // A new String object
var x3 = new Number();    // A new Number object
var x4 = new Boolean()    // A new Boolean object
var x5 = new Array();     // A new Array object
var x6 = new RegExp();    // A new RegExp object
var x7 = new Function();  // A new Function object
var x8 = new Date();      // A new Date object
```

Try it Yourself »

The Math() object is not in the list. Math is a global object. The new keyword cannot be used on Math.

Did You Know?

As you can see, JavaScript has object versions of the primitive data types String, Number, and Boolean.

There is no reason to create complex objects. Primitive values execute much faster.

And there is no reason to use new Array(). Use array literals instead: []

And there is no reason to use new RegExp(). Use pattern literals instead: /()/

And there is no reason to use new Function(). Use function expressions instead: function () {}.

And there is no reason to use new Object(). Use object literals instead: {}

Example

```
var x1 = {};            // new object
var x2 = "";            // new primitive string
var x3 = 0;             // new primitive number
var x4 = false;         // new primitive boolean
var x5 = [];            // new array object
var x6 = /()/           // new regexp object
var x7 = function(){};  // new function object
```

Try it Yourself »

JavaScript Objects are Mutable

Objects are mutable: They are addressed by reference, not by value.

If y is an object, the following statement will not create a copy of y:

```
var x = y;  // This will not create a copy of y.
```

The object x is not a **copy** of y. It **is** y. Both x and y points to the same object.

Any changes to y will also change x, because x and y are the same object.

Example

```
var person = {firstName:"John", lastName:"Doe", age:50,
eyeColor:"blue"}

var x = person;
x.age = 10;           // This will change both x.age and person.age
```

Try it Yourself »

« Previous Next Chapter »